

Ex 8: Deploying a Fine-Tuned Language Model for Customer Support with Evaluation (BLEU & ROUGE)

Learning Objective:

Develop a Python-based application to fine-tune and deploy a language model for customer support using platforms like Hugging Face Spaces or Streamlit, and evaluate its performance on the entire test dataset using BLEU and ROUGE metrics.

Steps:

1. Import Required Libraries

Import Transformers, Datasets, Torch, and evaluation libraries (BLEU, ROUGE).

2. Load Dataset

Use a Customer Service dataset or Bank FAQ dataset containing queries and responses.

3. Data Preprocessing

- Clean and normalize text
- Prepare input-output pairs (query → response)

4. Split Dataset

Divide into training and testing datasets.

5. Load Pre-trained Model

Load models such as T5 or BART from Hugging Face.

6. Fine-Tune the Model

Train the model using the training dataset.

7. Save the Fine-Tuned Model

Store trained model and tokenizer.

8. Build Inference Pipeline

Create a function to generate responses for input queries.

9. Generate Predictions on Test Dataset

- Pass all test queries to the model
- Generate responses for each query

10. Compute BLEU Scores (BLEU-1 to BLEU-4)

- Measure n-gram precision between generated and reference responses
- Compute BLEU-1, BLEU-2, BLEU-3, and BLEU-4 for each sample

11. Compute ROUGE Scores

- ROUGE-1 → unigram recall

- ROUGE-2 → bigram recall
- ROUGE-L for sequence similarity

12. Aggregate Evaluation Metrics

- Calculate average scores across all test samples
- Prepare overall performance summary

13. Display Evaluation Results

Metric	Score
BLEU-1	
BLEU-2	
BLEU-3	
BLEU-4	
ROUGE-1	
ROUGE-2	
ROUGE-L	

14. Develop User Interface

- Use Streamlit for web app
- OR deploy using Hugging Face Spaces

15. Deploy the Application

- Host the model for real-time customer query handling

16. Test with Sample Queries

Validate the deployed system with real-time user inputs.

17. Display Output

Show generated responses in the interface.

Program Code:

finetune_evaluate.py

```
# _____
# STEP 1: Import Required Libraries
# _____
```

```
import os
import json
import torch
import pandas as pd
from datasets import Dataset, DatasetDict
from transformers import (
    T5ForConditionalGeneration,
    T5Tokenizer,
    Seq2SeqTrainer,
    Seq2SeqTrainingArguments,
    DataCollatorForSeq2Seq,
)
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
from rouge_score import rouge_scorer
import nltk
import re
```

```
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
```

```
# _____
```

```
# STEP 2: Load Dataset
```

```
# _____
```

```
def load_dataset_from_csv(path: str) -> pd.DataFrame:
    """Load a CSV with 'query' and 'response' columns."""
    df = pd.read_csv(path)
    assert "query" in df.columns and "response" in df.columns, \
        "CSV must have 'query' and 'response' columns."
    return df
```

```

def load_bitext_dataset(max_samples: int = 5000) -> pd.DataFrame:
    """
    Load Bitext customer support dataset from HuggingFace.

    The dataset has 'instruction' (query) and 'response' columns.
    Limit to max_samples for faster training on smaller hardware.
    """
    try:
        print("📖 Loading Bitext customer support dataset from HuggingFace ...")
        from datasets import load_dataset as hf_load_dataset

        ds = hf_load_dataset("bitext/Bitext-customer-support-llm-chatbot-training-dataset")
        df = ds["train"].to_pandas()

        # Rename columns to match our pipeline
        df = df.rename(columns={"instruction": "query"})[["query", "response"]]

        # Limit to max_samples
        if len(df) > max_samples:
            print(f"⚠️ Limiting to {max_samples} samples (original: {len(df)})")
            df = df.sample(n=max_samples, random_state=42)

        print(f"✅ Loaded {len(df)} samples from Bitext dataset")
        return df
    except Exception as e:
        print(f"⚠️ Could not load Bitext dataset: {e}")
        print(" Falling back to sample dataset.")
        return load_sample_dataset()

```

```
def load_sample_dataset() -> pd.DataFrame:
    """
    Fallback: tiny built-in FAQ dataset so the script runs without
    an external file. Replace / extend with your real CSV.
    """
    data = [
        {"query": "How do I reset my password?",
         "response": "Click 'Forgot Password' on the login page and follow the instructions sent
         to your email."},
        {"query": "What are your business hours?",
         "response": "We are open Monday to Friday, 9 AM – 6 PM EST."},
        {"query": "How can I track my order?",
         "response": "Log in to your account and go to 'My Orders' to see real-time tracking."},
        {"query": "Can I return a product?",
         "response": "Yes, you can return any product within 30 days of purchase for a full
         refund."},
        {"query": "How do I contact customer support?",
         "response": "You can reach us via live chat, email at support@example.com, or call 1-
         800-EXAMPLE."},
        {"query": "Do you offer international shipping?",
         "response": "Yes, we ship to over 50 countries. Shipping costs depend on your
         location."},
        {"query": "How do I update my billing information?",
         "response": "Go to Account Settings > Payment Methods and update your card
         details."},
        {"query": "What payment methods do you accept?",
         "response": "We accept Visa, Mastercard, American Express, PayPal, and Apple Pay."},
        {"query": "How do I cancel my subscription?",
         "response": "Navigate to Account Settings > Subscription and click 'Cancel
         Subscription'."}]
```

```
        {"query": "Is my personal data secure?",
         "response": "We use 256-bit SSL encryption and never share your data with third
parties."},
    ]
    return pd.DataFrame(data)
```

```
# _____
```

```
# STEP 3: Data Preprocessing
```

```
# _____
```

```
def clean_text(text: str) -> str:
```

```
    """Lowercase, strip extra whitespace, remove special characters."""
```

```
    text = text.lower().strip()
```

```
    text = re.sub(r"\s+", " ", text)
```

```
    text = re.sub(r"^[^w\s?!.',-]", "", text)
```

```
    return text
```

```
def preprocess(df: pd.DataFrame) -> pd.DataFrame:
```

```
    df = df.copy()
```

```
    df["query"] = df["query"].apply(clean_text)
```

```
    df["response"] = df["response"].apply(clean_text)
```

```
    df.dropna(subset=["query", "response"], inplace=True)
```

```
    df.drop_duplicates(subset=["query"], inplace=True)
```

```
    return df
```

```
# _____
```

```
# STEP 4: Split Dataset
```

```
# _____
```

```
def split_dataset(df: pd.DataFrame, test_size: float = 0.2, seed: int = 42):
```

```
    df = df.sample(frac=1, random_state=seed).reset_index(drop=True)
```

```
split = int(len(df) * (1 - test_size))
train_df = df.iloc[:split]
test_df = df.iloc[split:]
print(f"Train samples: {len(train_df)} | Test samples: {len(test_df)}")
return train_df, test_df
```

```
# _____
```

```
# STEP 5 & 6: Load Pre-trained Model & Fine-Tune
```

```
# _____
```

```
MODEL_NAME = "t5-small"    # swap for "facebook/bart-base" etc.
```

```
MAX_INPUT = 128
```

```
MAX_TARGET = 128
```

```
BATCH_SIZE = 4
```

```
EPOCHS = 3
```

```
OUTPUT_DIR = "./fine_tuned_model"
```

```
def tokenize_batch(examples, tokenizer):
```

```
    inputs = tokenizer(
        ["customer support: " + q for q in examples["query"]],
        max_length=MAX_INPUT, truncation=True, padding="max_length",
    )
    with tokenizer.as_target_tokenizer():
        targets = tokenizer(
            examples["response"],
            max_length=MAX_TARGET, truncation=True, padding="max_length",
        )
    inputs["labels"] = [
        [(t if t != tokenizer.pad_token_id else -100) for t in label]
        for label in targets["input_ids"]
    ]
```

```
]
```

```
return inputs
```

```
def fine_tune(train_df: pd.DataFrame, test_df: pd.DataFrame):
```

```
    tokenizer = T5Tokenizer.from_pretrained(MODEL_NAME)
```

```
    model = T5ForConditionalGeneration.from_pretrained(MODEL_NAME)
```

```
    # Convert to HuggingFace Dataset
```

```
    train_hf = Dataset.from_pandas(train_df[["query", "response"]])
```

```
    test_hf = Dataset.from_pandas(test_df[["query", "response"]])
```

```
    raw = DatasetDict({"train": train_hf, "test": test_hf})
```

```
    tokenized = raw.map(
```

```
        lambda ex: tokenize_batch(ex, tokenizer),
```

```
        batched=True, remove_columns=["query", "response"],
```

```
    )
```

```
    args = Seq2SeqTrainingArguments(
```

```
        output_dir=OUTPUT_DIR,
```

```
        num_train_epochs=EPOCHS,
```

```
        per_device_train_batch_size=BATCH_SIZE,
```

```
        per_device_eval_batch_size=BATCH_SIZE,
```

```
        warmup_steps=10,
```

```
        weight_decay=0.01,
```

```
        logging_dir="./logs",
```

```
        logging_steps=10,
```

```
        evaluation_strategy="epoch",
```

```
        save_strategy="epoch",
```

```
        load_best_model_at_end=True,
```



```
        predict_with_generate=True,  
        fp16=torch.cuda.is_available(),  
    )
```

```
collator = DataCollatorForSeq2Seq(tokenizer, model=model, padding=True)
```

```
trainer = Seq2SeqTrainer(  
    model=model,  
    args=args,  
    train_dataset=tokenized["train"],  
    eval_dataset=tokenized["test"],  
    tokenizer=tokenizer,  
    data_collator=collator,  
)
```

```
print("\n🚀 Starting fine-tuning ...")
```

```
trainer.train()
```

```
# _____
```

```
# STEP 7: Save the Fine-Tuned Model
```

```
# _____
```

```
model.save_pretrained(OUTPUT_DIR)
```

```
tokenizer.save_pretrained(OUTPUT_DIR)
```

```
print(f"✅ Model saved to {OUTPUT_DIR}")
```

```
return model, tokenizer
```

```
# _____
```

```
# STEP 8: Inference Pipeline
```

```
# -----  
  
def generate_response(query: str, model, tokenizer, max_len: int = 128) -> str:  
    input_text = "customer support: " + query.lower().strip()  
    inputs = tokenizer(  
        input_text, return_tensors="pt",  
        max_length=MAX_INPUT, truncation=True,  
    ).to(model.device)  
    outputs = model.generate(  
        **inputs,  
        max_length=max_len,  
        num_beams=4,  
        early_stopping=True,  
    )  
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
# -----  
# STEP 9: Generate Predictions on Test Dataset  
# -----
```

```
def predict_all(test_df: pd.DataFrame, model, tokenizer) -> pd.DataFrame:  
    print("\n 📄 Generating predictions ...")  
    model.eval()  
    preds = [generate_response(q, model, tokenizer) for q in test_df["query"]]  
    result = test_df.copy()  
    result["predicted"] = preds  
    return result
```

```
# -----  
# STEP 10: Compute BLEU Scores  
# -----
```

```

def compute_bleu(reference: str, hypothesis: str):
    ref_tokens = nltk.word_tokenize(reference)
    hyp_tokens = nltk.word_tokenize(hypothesis)
    smooth = SmoothingFunction().method1
    weights_map = {
        "BLEU-1": (1, 0, 0, 0),
        "BLEU-2": (0.5, 0.5, 0, 0),
        "BLEU-3": (1/3, 1/3, 1/3, 0),
        "BLEU-4": (0.25, 0.25, 0.25, 0.25),
    }
    return {
        k: sentence_bleu([ref_tokens], hyp_tokens, weights=w, smoothing_function=smooth)
        for k, w in weights_map.items()
    }

```

STEP 11: Compute ROUGE Scores

```

def compute_rouge(reference: str, hypothesis: str):
    scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"], use_stemmer=True)
    scores = scorer.score(reference, hypothesis)
    return {
        "ROUGE-1": scores["rouge1"].fmeasure,
        "ROUGE-2": scores["rouge2"].fmeasure,
        "ROUGE-L": scores["rougeL"].fmeasure,
    }

```

STEP 12 & 13: Aggregate & Display Metrics

```
# -----  
  
def evaluate(predictions_df: pd.DataFrame) -> dict:  
    all_scores = []  
    for _, row in predictions_df.iterrows():  
        bleu = compute_bleu(row["response"], row["predicted"])  
        rouge = compute_rouge(row["response"], row["predicted"])  
        all_scores.append(**{bleu, rouge})  
  
    scores_df = pd.DataFrame(all_scores)  
    avg = scores_df.mean().to_dict()  
  
    print("\n" + "=" * 50)  
    print("    EVALUATION RESULTS (Test Set Averages)")  
    print("=" * 50)  
    print(f'{"Metric":<12} {"Score":>10}')  
    print("-" * 25)  
    for metric, val in avg.items():  
        print(f'{metric:<12} {val:>10.4f}')  
    print("=" * 50)  
  
    # Save results  
    results_path = os.path.join(OUTPUT_DIR, "evaluation_results.json")  
    with open(results_path, "w") as f:  
        json.dump(avg, f, indent=2)  
    print(f"\n📁 Results saved to {results_path}")  
  
    return avg  
  
# -----
```

```
# MAIN

# -----

if __name__ == "__main__":

    # --- Load ---

    CSV_PATH = "customer_support_data.csv"

    print("=" * 60)

    print("    CUSTOMER SUPPORT NLP — DATASET LOADING")

    print("=" * 60)

    if os.path.exists(CSV_PATH):

        print(f"\n✓ Found {CSV_PATH}, loading ...")

        df = load_dataset_from_csv(CSV_PATH)

    else:

        print("\n✓ Attempting to load Bitext dataset from HuggingFace ...")

        df = load_bitext_dataset(max_samples=5000)

    print(f"Dataset size: {len(df)} samples\n")

    # --- Preprocess & Split ---

    df = preprocess(df)

    train_df, test_df = split_dataset(df)

    # --- Fine-Tune ---

    model, tokenizer = fine_tune(train_df, test_df)

    # --- Predict ---

    predictions_df = predict_all(test_df, model, tokenizer)

    predictions_df.to_csv(os.path.join(OUTPUT_DIR, "predictions.csv"), index=False)
```

```
# --- Evaluate ---
```

```
metrics = evaluate(predictions_df)
```

app.py

```
"""
```

Ex 8 — Steps 14–17: Streamlit UI for Customer Support Chatbot

Run: streamlit run app.py

```
"""
```

```
import os
```

```
import json
```

```
import torch
```

```
import streamlit as st
```

```
from transformers import T5ForConditionalGeneration, T5Tokenizer
```

```
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
```

```
from rouge_score import rouge_scorer
```

```
import nltk
```

```
import pandas as pd
```

```
nltk.download("punkt", quiet=True)
```

```
nltk.download("punkt_tab", quiet=True)
```

```
# _____
```

```
# Config
```

```
# _____
```

```
MODEL_DIR = "./fine_tuned_model"
```

```
FALLBACK = "t5-small"      # used if fine-tuned model not found
```

```
MAX_INPUT = 128
```

MAX_TARGET = 128

Page Setup

st.set_page_config(

 page_title="Customer Support AI",

 page_icon="🤖",

 layout="wide",

)

st.markdown("""

<style>

 /* — global — */

 @import

url('https://fonts.googleapis.com/css2?family=Space+Mono:wght@400;700&family=DM+Sans:wght@300;400;600&display=swap');

html, body, [class*="css"] { font-family: 'DM Sans', sans-serif; }

.main { background: #0f1117; }

/* — header — */

.hero {

 background: linear-gradient(135deg, #1a1f2e 0%, #0d1b2a 100%);

 border: 1px solid #2a3347;

 border-radius: 16px;

 padding: 2rem 2.5rem;

 margin-bottom: 1.5rem;

 display: flex;

```
    align-items: center;
    gap: 1rem;
}

.hero h1 {
    font-family: 'Space Mono', monospace;
    font-size: 1.8rem;
    color: #e8f4fd;
    margin: 0;
}

.hero p { color: #7a8fa6; margin: 0.25rem 0 0; font-size: 0.9rem; }

/* — cards — */

.metric-card {
    background: #1a1f2e;
    border: 1px solid #2a3347;
    border-radius: 12px;
    padding: 1rem 1.25rem;
    text-align: center;
}

.metric-label {
    font-family: 'Space Mono', monospace;
    font-size: 0.7rem;
    color: #4a90d9;
    letter-spacing: 0.1em;
    text-transform: uppercase;
}

.metric-value {
    font-size: 1.6rem;
    font-weight: 600;
```



```
    color: #e8f4fd;
    margin-top: 0.2rem;
}

/* — chat bubbles — */
.bubble-user {
    background: #1e3a5f;
    border-radius: 16px 16px 4px 16px;
    padding: 0.75rem 1rem;
    margin: 0.5rem 0;
    color: #cde4f8;
    max-width: 80%;
    margin-left: auto;
}

.bubble-bot {
    background: #1a2535;
    border: 1px solid #2a3347;
    border-radius: 16px 16px 16px 4px;
    padding: 0.75rem 1rem;
    margin: 0.5rem 0;
    color: #d1dde8;
    max-width: 80%;
}

.timestamp {
    font-size: 0.68rem;
    color: #4a5568;
    margin-top: 0.3rem;
}
```

```

/* — input tweaks — */
textarea { background: #1a1f2e !important; color: #e8f4fd !important; }

.stButton > button {
    background: linear-gradient(135deg, #2563eb, #1d4ed8);
    color: white;
    border: none;
    border-radius: 8px;
    padding: 0.5rem 1.5rem;
    font-family: 'Space Mono', monospace;
    font-size: 0.8rem;
    letter-spacing: 0.05em;
    width: 100%;
}

.stButton > button:hover { background: linear-gradient(135deg, #3b82f6, #2563eb); }

/* hide streamlit chrome */
#MainMenu, footer, header { visibility: hidden; }
</style>
"", unsafe_allow_html=True)

# _____
# Load Model (cached)
# _____

@st.cache_resource(show_spinner="🔄 Loading model ...")
def load_model():
    path = MODEL_DIR if os.path.exists(MODEL_DIR) else FALLBACK

    try:
        tokenizer = T5Tokenizer.from_pretrained(path)

```

```

model = T5ForConditionalGeneration.from_pretrained(path)

# Use GPU if available, else CPU
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
model.eval()

return model, tokenizer, path, device
except Exception as e:
    st.error(f"❌ Error loading model from '{path}': {str(e)}")
    st.info("Make sure you've run `python finetune_evaluate.py` first to train the model.")
    st.stop()

model, tokenizer, loaded_from, device = load_model()

# _____
# Inference
# _____

def generate_response(query: str) -> str:
    """Generate a customer support response for the given query."""
    input_text = "customer support: " + query.strip().lower()
    inputs = tokenizer(
        input_text, return_tensors="pt",
        max_length=MAX_INPUT, truncation=True,
    ).to(device) # Move to the correct device

    with torch.no_grad():
        outputs = model.generate(
            **inputs,

```

```

        max_length=MAX_TARGET,
        num_beams=4,
        early_stopping=True,
    )
    return tokenizer.decode(outputs[0], skip_special_tokens=True)

# -----
# Evaluation helpers
# -----

def compute_bleu(reference: str, hypothesis: str) -> dict:
    ref = nltk.word_tokenize(reference)
    hyp = nltk.word_tokenize(hypothesis)
    sm = SmoothingFunction().method1
    return {
        "BLEU-1": sentence_bleu([ref], hyp, weights=(1,0,0,0), smoothing_function=sm),
        "BLEU-2": sentence_bleu([ref], hyp, weights=(.5,.5,0,0), smoothing_function=sm),
        "BLEU-3": sentence_bleu([ref], hyp, weights=(1/3,1/3,1/3,0), smoothing_function=sm),
        "BLEU-4": sentence_bleu([ref], hyp, weights=(.25,.25,.25,.25),
        smoothing_function=sm),
    }

def compute_rouge(reference: str, hypothesis: str) -> dict:
    sc = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"], use_stemmer=True)
    s = sc.score(reference, hypothesis)
    return {
        "ROUGE-1": s["rouge1"].fmeasure,
        "ROUGE-2": s["rouge2"].fmeasure,
        "ROUGE-L": s["rougeL"].fmeasure,
    }

```

```

# -----
# Layout — Header
# -----

st.markdown(f"""
<div class="hero">

    <div>

        <h1>🤖 Customer Support AI</h1>

        <p>Powered by fine-tuned T5 · Model: <code>{loaded_from}</code> · Device:
<code>{device.upper()}</code></p>

    </div>

</div>

""", unsafe_allow_html=True)

tab_chat, tab_eval, tab_batch = st.tabs(["💬 Chat", "📊 Evaluate", "📁 Batch Test"])

# =====
# TAB 1 — Chat (Steps 16 & 17)
# =====

with tab_chat:

    st.markdown("""

    ### Chat with the AI

    Ask customer support questions and get instant responses powered by your fine-tuned T5
    model.

    """)

    if "history" not in st.session_state:

        st.session_state.history = []

```

```

# Display history in a container with scrolling
history_container = st.container(border=True)

with history_container:
    if st.session_state.history:
        for turn in st.session_state.history:
            st.markdown(f<div class="bubble-user"> 🧑 {turn["query"]} </div>',
unsafe_allow_html=True)

            st.markdown(f<div class="bubble-bot"> 🤖 {turn["response"]} </div>',
unsafe_allow_html=True)
        else:
            st.info("💬 Start a conversation by typing a question below ...")

# Input section
st.markdown("---")
col1, col2 = st.columns([5, 1])

with col1:
    user_input = st.text_area(
        "Your query",
        placeholder="e.g. How do I reset my password?",
        label_visibility="collapsed",
        height=80,
        key="chat_input"
    )

with col2:
    st.write("")
    st.write("")

    send = st.button("Send ➤", use_container_width=True)

if send and user_input.strip():

```

```
with st.spinner("🤖 Generating response ..."):
```

```
    try:
```

```
        resp = generate_response(user_input)
```

```
        st.session_state.history.append({"query": user_input, "response": resp})
```

```
        st.rerun()
```

```
    except Exception as e:
```

```
        st.error(f"❌ Error generating response: {str(e)}")
```

```
# Clear chat button
```

```
if st.session_state.history:
```

```
    if st.button("🗑️ Clear Chat", use_container_width=True):
```

```
        st.session_state.history = []
```

```
        st.rerun()
```

```
# =====
```

```
# TAB 2 — Evaluate (Steps 10–13)
```

```
# =====
```

```
with tab_eval:
```

```
    st.subheader("📊 Evaluation Metrics")
```

```
    st.markdown("""
```

```
    Compare reference (ground truth) responses with generated responses.
```

```
    **BLEU** measures n-gram precision, **ROUGE** measures recall similarity.
```

```
    """)
```

```
    col_ref, col_hyp = st.columns(2)
```

```
    with col_ref:
```

```
        st.markdown("***Reference Response** (ground truth)")
```

```
        ref_text = st.text_area(
```

```
    "Reference",
    height=120,
    placeholder="The expected correct answer ...",
    label_visibility="collapsed",
    key="ref_input"
)
```

with col_hyp:

```
    st.markdown("**Generated Response** (model output)")
    hyp_text = st.text_area(
        "Generated",
        height=120,
        placeholder="What the model produced ...",
        label_visibility="collapsed",
        key="hyp_input"
    )
```

if st.button("Compute Metrics", use_container_width=True):

if ref_text.strip() and hyp_text.strip():

try:

```
            bleu = compute_bleu(ref_text, hyp_text)
            rouge = compute_rouge(ref_text, hyp_text)
            all_m = {**bleu, **rouge}
```

```
    st.markdown("---")
```

```
    st.markdown("###  Computed Scores")
```

```
    cols = st.columns(4)
```

```
    for col, (k, v) in zip(cols * 2, all_m.items()):
```

```
        with col:
```



```

st.markdown(f"""
<div class="metric-card">

    <div class="metric-label">{k}</div>

    <div class="metric-value">{v:.4f}</div>

</div>""", unsafe_allow_html=True)

# Saved results from training
results_path = os.path.join(MODEL_DIR, "evaluation_results.json")
if os.path.exists(results_path):
    with open(results_path) as f:
        saved = json.load(f)
    st.markdown("---")
    st.markdown("### 📄 Test-set Averages (from fine-tuning)")
    df = pd.DataFrame(saved.items(), columns=["Metric", "Score"])
    df["Score"] = df["Score"].apply(lambda x: f"{x:.4f}")
    st.dataframe(df, use_container_width=True, hide_index=True)
except Exception as e:
    st.error(f"❌ Error computing metrics: {str(e)}")
else:
    st.warning(f"⚠️ Please fill in both reference and generated responses.")

# =====
# TAB 3 — Batch Test (Steps 9 & 16)
# =====

with tab_batch:
    st.subheader("📁 Batch Processing")
    st.markdown("""

    Upload a CSV file with customer queries. The model will generate responses for all of
    them.

```

****CSV Format:****

- ****Required:**** `query` column with customer questions
 - ****Optional:**** `response` column with ground truth for evaluation metrics
- """)

uploaded = st.file_uploader("Choose a CSV file", type=["csv"])

if uploaded:

try:

df = pd.read_csv(uploaded)

st.markdown("---")

st.markdown(f"****Loaded:**** {len(df)} rows")

st.dataframe(df.head(), use_container_width=True)

if "query" not in df.columns:

st.error("❌ CSV must contain a `query` column.")

else:

st.markdown("---")

if st.button("🚀 Run Batch Inference", use_container_width=True):

progress_bar = st.progress(0)

status_text = st.empty()

try:

Generate predictions

predictions = []

for idx, query in enumerate(df["query"]):

status_text.text(f"Processing {idx + 1} of {len(df)} ...")

pred = generate_response(query)

```

        predictions.append(pred)
        progress_bar.progress((idx + 1) / len(df))

df["predicted"] = predictions

# Compute metrics if reference responses exist
if "response" in df.columns:
    status_text.text("Computing BLEU & ROUGE scores ...")
    bleu_scores = df.apply(
        lambda r: compute_bleu(r["response"], r["predicted"]), axis=1)
    rouge_scores = df.apply(
        lambda r: compute_rouge(r["response"], r["predicted"]), axis=1)

    metrics_df = pd.DataFrame(list(bleu_scores)) \
        .join(pd.DataFrame(list(rouge_scores)))
    df = pd.concat([df, metrics_df], axis=1)

progress_bar.progress(1.0)
status_text.text("✅ Complete!")

st.markdown("---")
st.markdown("### Results")
st.dataframe(df, use_container_width=True)

# Download button
csv_out = df.to_csv(index=False).encode()
st.download_button(
    label="⬇ Download Results as CSV",
    data=csv_out,

```

```

        file_name="batch_results.csv",

        mime="text/csv",

        use_container_width=True

    )

except Exception as e:

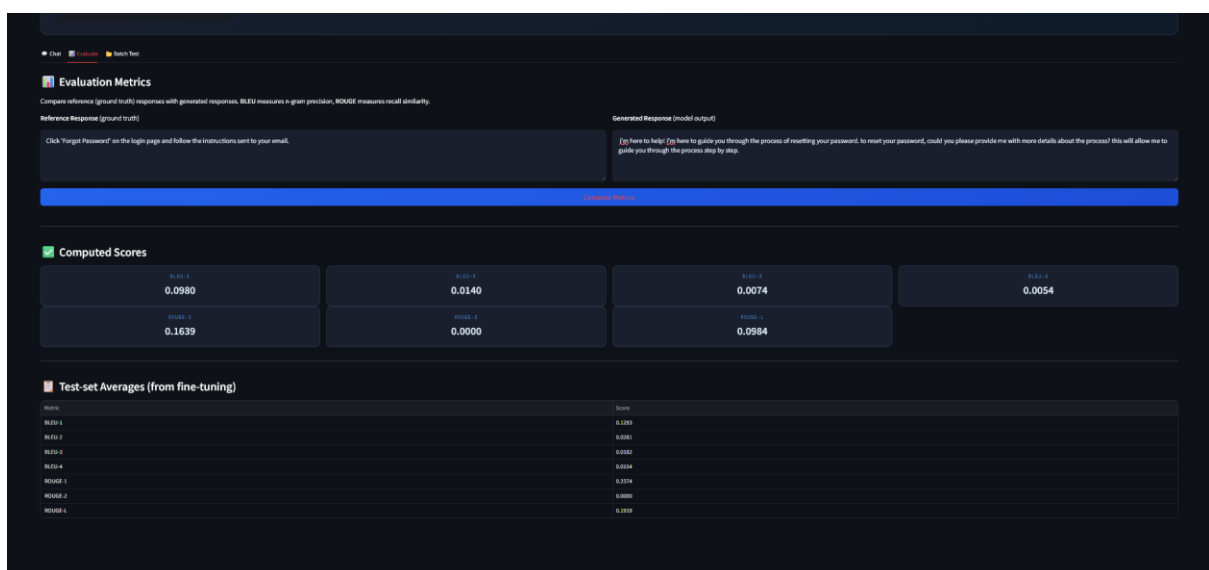
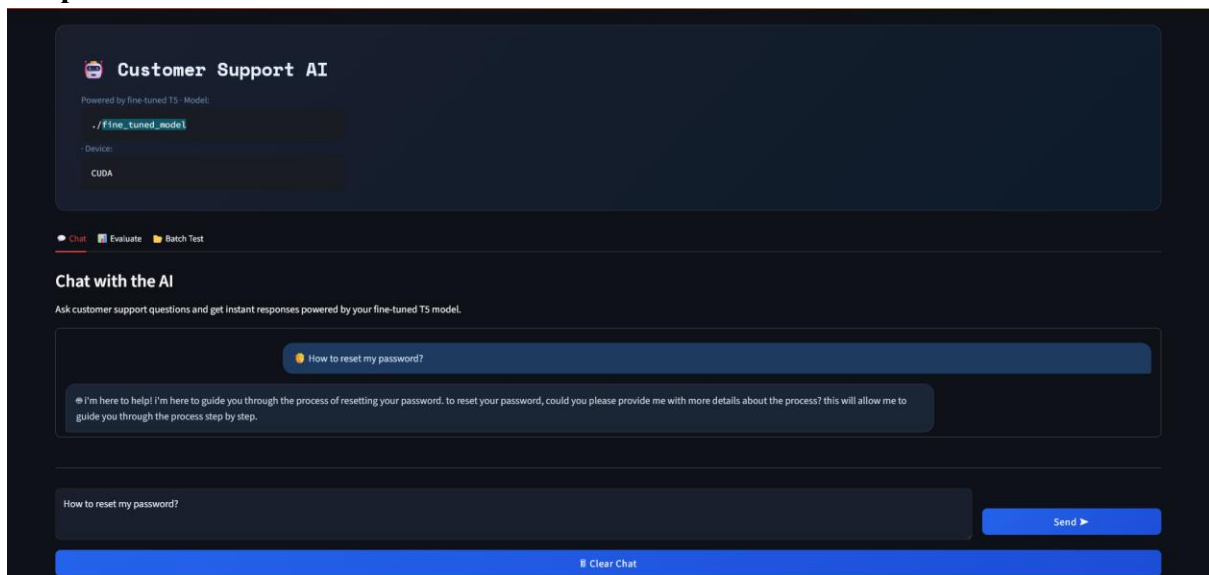
    st.error(f"❌ Error during batch processing: {str(e)}")

except Exception as e:

    st.error(f"❌ Error reading CSV: {str(e)}")

```

Output:



ChatEvaluateBatch Test

Batch Processing

Upload a CSV file with customer queries. The model will generate responses for all of them.

CSV Format:

- Required: `query` column with customer questions
- Optional: `response` column with ground truth for evaluation metrics

Choose a CSV file

Drag and drop file here
Limit 200MB per file • CSV

example_queries.csv 2.2KB

Loaded: 20 rows

| query | response |
|--------------------------------------|---|
| 0 How do I reset my password? | Click 'Forgot Password' on the login page and follow the instructions sent to your email. |
| 1 What are your business hours? | We are open Monday to Friday from 9 AM to 6 PM EST. |
| 2 How can I track my order? | Log in to your account and go to 'My Orders' to see real time tracking information. |
| 3 Can I return a product? | Yes you can return any product within 30 days of purchase for a full refund. |
| 4 How do I contact customer support? | You can reach us via live chat email at support@example.com or call 1-800-EXAMPLE. |

Run Batch Inference

| query | response | predicted | BLEU-1 | BLEU-2 | BLEU-3 | BLEU-4 | ROUGE-1 | ROUGE-2 | ROUGE-L |
|--|---|---|--------|--------|--------|--------|---------|---------|---------|
| 0 How do I reset my password? | Click 'Forgot Password' on the login page and follow the instructions sent to your email. | I'm here to help! I'm here to guide you through the process of resetting your password. I | 0.098 | 0.014 | 0.0074 | 0.0054 | 0.1639 | 0 | 0.0984 |
| 1 What are your business hours? | We are open Monday to Friday from 9 AM to 6 PM EST. | thank you for reaching out! I'm here to help! I'm here to help! I'm here to guide you thro | 0.0566 | 0.0104 | 0.006 | 0.0045 | 0.0656 | 0 | 0.0656 |
| 2 How can I track my order? | Log in to your account and go to 'My Orders' to see real time tracking information. | thank you for reaching out! I'm here to assist you in tracking your order, to track your or | 0.1373 | 0.0166 | 0.0082 | 0.0058 | 0.2258 | 0.0333 | 0.1613 |
| 3 Can I return a product? | Yes you can return any product within 30 days of purchase for a full refund. | thank you for reaching out! I'm here to guide you through the process of returning a prc | 0.1061 | 0.0128 | 0.0063 | 0.0045 | 0.1379 | 0 | 0.1063 |
| 4 How do I contact customer support? | You can reach us via live chat email at support@example.com or call 1-800-EXAMPLE. | thank you for reaching out! I'm here to help! I'm here to help! I'm here to help! I'm here | 0 | 0 | 0 | 0 | 0.0851 | 0 | 0.0851 |
| 5 Do you offer international shipping? | Yes we ship to over 50 countries. Shipping costs depend on your location. | I'm here to help! I'm here to help! I'm here to help! I'm here to guide you through the pr | 0.0541 | 0.0123 | 0.0075 | 0.006 | 0.0889 | 0 | 0.0889 |
| 6 How do I update my billing information? | Go to Account Settings > Payment Methods and update your card details. | thank you for reaching out! I'm here to assist you in updating your billing information. t | 0.0893 | 0.0403 | 0.0144 | 0.0087 | 0.129 | 0.0333 | 0.129 |
| 7 What payment methods do you accept? | We accept Visa Mastercard American Express PayPal and Apple Pay. | thank you for reaching out! I'm here to help! I'm here to guide you through the process | 0.0429 | 0.0079 | 0.0045 | 0.0034 | 0.0541 | 0 | 0.0541 |
| 8 How do I cancel my subscription? | Navigate to Account Settings > Subscription and click 'Cancel Subscription'. | I'm here to help! I'm here to assist you in canceling your subscription, to cancel your su | 0.038 | 0.007 | 0.004 | 0.003 | 0.15 | 0.0256 | 0.1 |
| 9 Is my personal data secure? | We use 256-bit SSL encryption and never share your data with third parties. | thank you for reaching out! I'm here to guide you through the process of storing your pe | 0.0926 | 0.0132 | 0.007 | 0.0051 | 0.127 | 0 | 0.0952 |
| 10 What is your return policy? | We offer a 30-day money-back guarantee on all purchases no questions asked. | thank you for reaching out! I'm here to guide you through the process and ensure a smc | 0.0317 | 0.0072 | 0.0044 | 0.0034 | 0.0278 | 0 | 0.0278 |
| 11 How long does shipping take? | Standard shipping takes 5-7 business days. Express shipping is available for 1-2 day deli | I'm here to help! I'm here to guide you through the process and ensure a smooth shippi | 0.05 | 0.008 | 0.0043 | 0.0032 | 0.0449 | 0 | 0.0449 |
| 12 Can I change my order after placing it? | If your order hasn't shipped yet you can modify it within 2 hours of placement. | I'm here to guide you through the process of changing your order after placing it, to cha | 0.0952 | 0.0392 | 0.0136 | 0.008 | 0.1351 | 0.0278 | 0.0811 |
| 13 Do you offer discounts for bulk orders? | Yes please contact our sales team at sales@example.com for volume pricing. | thank you for reaching out! I'm here to assist you in offering discounts for bulk orders. b | 0.0508 | 0.0094 | 0.0054 | 0.0041 | 0.0597 | 0 | 0.0597 |
| 14 What warranty do you provide? | All products come with a 1-year manufacturer warranty covering defects. | thank you for reaching out! I'm here to help! I'm here to guide you through the process. | 0.0435 | 0.008 | 0.0046 | 0.0035 | 0.0541 | 0 | 0.0541 |
| 15 How do I apply a discount code? | Enter your code in the 'Promo Code' field during checkout before completing your purc | thank you for reaching out! I'm here to guide you through the process of applying a disc | 0.0492 | 0.0091 | 0.0052 | 0.0039 | 0.0857 | 0 | 0.0857 |
| 16 Can I get an invoice for my purchase? | Yes invoices are automatically emailed to you after your order is confirmed. | thank you for reaching out! I'm here to assist you in getting a invoice for your purchase. | 0.0625 | 0.01 | 0.0054 | 0.004 | 0.1127 | 0 | 0.1127 |
| 17 What is your privacy policy? | We protect your data and never share personal information with third parties without c | I'm here to help! I'm here to guide you through the process of resolving your privacy issi | 0.0563 | 0.009 | 0.0049 | 0.0036 | 0.1013 | 0 | 0.0759 |
| 18 How do I update my account information? | You can update your profile details in Account Settings > Profile Information. | thank you for reaching out! I'm here to guide you through the process of updating your | 0.0678 | 0.0342 | 0.0127 | 0.0078 | 0.1846 | 0.0317 | 0.1538 |
| 19 Do you have a loyalty program? | Yes join our rewards program to earn points on every purchase for future discounts. | thank you for reaching out! I'm here to help! I'm here to help! I'm here to guide you thro | 0.0556 | 0.0088 | 0.0048 | 0.0036 | 0.0759 | 0.026 | 0.0506 |

Learning Outcome:

Students will be able to fine-tune, evaluate, and deploy a language model for customer support systems. They will gain practical experience in computing **BLEU and ROUGE scores**, enabling proper performance assessment of NLP models in real-world applications.